

AdiVaani Initiative — MISN Lab, IIT Delhi

Technical Report: Part I

Classical Neural Machine Translation

Hindi ↔ Marathi

LSTM Seq2Seq with Bahdanau Attention

https://github.com/1KartikeyaJain/Adivaani_Kartikeya

Submitted By
Kartikeya Jain

1. Overview and Research Motivation

This report describes the design, implementation, training and analysis of a complete Neural Machine Translation (NMT) system for Hindi–Marathi translation, of the hiring assessment.

Hindi and Marathi are both Indo-Aryan languages with substantial morphological structure, script (Devanagari), and grammatical patterns, making them an interesting testbed for NMT systems. Although the two languages are related, they have significant differences in vocabulary, inflectional paradigms and syntactic constructions that make machine translation not trivial.

In this part we investigate the central research question: how does the choice of the embedding strategy randomly initialised versus pretrained BERT based contextual representations affect the convergence, translation quality and generalisation behaviour of an LSTM based Seq2Seq system? The scientific core of the report is this comparative analysis.

2. Dataset and Preprocessing

2.1 Corpus Description

The Hindi–Marathi parallel corpus provided for this assignment was accessed through Kaggle. The dataset contains aligned sentence pairs across the two languages in separate .hi (Hindi) and .mr (Marathi) text files. Two separate Kaggle notebooks were used for the two experiments:

- Notebook 1 (Random Embeddings)
- Notebook 2 (BERT Embeddings)

Both datasets point to the same underlying Hindi–Marathi parallel corpus from the AdiVaani assignment.

2.2 Data Filtering and Length Constraints

Raw parallel data was filtered to retain only sentence pairs where both sides contained between 1 and 60 tokens (word-level) for the random embeddings experiment, and between 1 and 50 tokens for the BERT experiment (the tighter constraint for BERT was applied due to BERT's maximum sequence length of 512 WordPiece tokens and to control memory usage). The filtering pipeline operated as follows:

1. Read aligned .hi and .mr file pairs line by line
2. Strip leading/trailing whitespace; discard empty lines
3. Apply whitespace-level token count filter per side
4. Randomly sample up to the configured maximum (300,000 for random; 200,000 for BERT)
5. Shuffle and split 95% train / 5% validation

Parameter	Random Embeddings	BERT Embeddings
Max sentence length (words)	60	50
Max samples used	300,000	200,000

Parameter	Random Embeddings	BERT Embeddings
Val split	5%	5%
Approx. train pairs	~285,000	~190,000
Approx. val pairs	~15,000	~10,000

Claude was consulted when designing the data loading pipeline to ensure the filtering logic handled edge cases such as mismatched line counts and empty segments correctly.

2.3 Tokenization Strategy

A key early design decision was tokenization. Three options considered:

- Character-level tokenization: maximally handles OOV but produces very long sequences, making LSTM training slow and degrading gradient flow.
- Word-level tokenization: simple but produces large vocabularies with severe OOV problems for morphologically rich languages like Marathi.
- Subword tokenization (BPE / SentencePiece): balances vocabulary size against OOV coverage, handles morphological variants naturally.

SentencePiece BPE tokenization was chosen for both experiments. For the random embeddings experiment training of shared vocabulary of 8,000 SentencePiece BPE tokens on both Hindi and Marathi data was done. This was done because Hindi and Marathi use the Devanagari script and they have a lot of similar SentencePiece BPE tokens. Using a shared vocabulary lets the model learn things that're common to both Hindi and Marathi at the SentencePiece BPE token level.

This means the model has things to learn and it can use the same information, for both Hindi and Marathi SentencePiece BPE tokens. For the BERT experiment, the source side is tokenized by BERT's own WordPiece tokenizer (separate tokenizers for Hindi BERT and Marathi BERT), while the target side uses a dedicated SentencePiece BPE model trained solely on Marathi data (8,000 tokens). This design separates the concerns of representation (handled by BERT) and generation (handled by the SPM-based decoder vocabulary).

SentencePiece was configured with `character_coverage=0.9995`, ensuring that almost all Unicode characters in both scripts are handled, with a small UNK budget for extremely rare characters. Special tokens were set as: PAD=0, UNK=1, BOS=2, EOS=3.

3. Model Architecture

3.1 General Seq2Seq Framework

Both experiments share the same encoder-decoder Seq2Seq framework with Bahdanau (additive) attention, following the architecture introduced by Bahdanau et al. (2015). The key difference lies exclusively in how the source-side tokens are mapped to continuous representations before entering the encoder LSTM.

3.2 Encoder

The encoder is a bidirectional 2-layer LSTM. Bidirectionality is critical for encoder representations: the forward LSTM sees context from the left, and the backward LSTM from the right, providing each token with full sentence context before attention is computed. The encoder outputs:

- `enc_out`: a tensor of shape (B, S, 2H) — the concatenated hidden states from both directions at every position, used as keys/values in the attention mechanism.
- (`h_init`, `c_init`): initialisation states for the decoder, derived by projecting the final bidirectional hidden state through a learned linear layer followed by tanh.

For the random embeddings model, an `nn.Embedding` layer (256-dim) maps token IDs to dense vectors before being fed into the encoder LSTM. For the BERT model, pre-computed BERT contextual embeddings (768-dim) are projected down to 256-dim through a learned projection block before entering the same LSTM structure.

3.3 Bahdanau Attention

The attention mechanism follows the additive formulation from Bahdanau et al. (2015). At each decoder step, the attention module computes:

- An energy score between the current decoder hidden state and each encoder output position, via: $\text{energy} = \tanh(W_{\text{enc}} \times \text{enc_out} + W_{\text{dec}} \times \text{dec_h})$
- A scalar score per position via a learned projection v , followed by softmax normalisation to obtain attention weights.
- A context vector as the weighted sum of encoder outputs.

Source padding is masked out before softmax, preventing the model from attending to padded positions. This is important for variable-length batches.

3.4 Decoder

The decoder is a 2-layer unidirectional LSTM. At each step it receives:

- The embedding of the previous target token (teacher-forced during training)
- The attention context vector from the encoder

These are concatenated and fed into the decoder LSTM. The output prediction follows a deep output approach : the decoder hidden state, context vector, and input embedding are concatenated and passed through a linear layer to produce logits over the target vocabulary. This residual-style combination has empirically shown to improve gradient flow during training.

3.5 BERT Embedding Integration

For the BERT experiment, pre-trained contextual embeddings were used as a replacement for the learned embedding table on the source side. The integration was designed with the following considerations:

Hindi sentences were encoded using `l3cube-pune/hindi-bert-v2` and Marathi sentences using `l3cube-pune/marathi-bert-v2`, both loaded from HuggingFace via the `transformers` library. BERT was operated in frozen mode (`bert_freeze=True`), meaning its weights were not updated during training. This decision was made to:

- Avoid catastrophic forgetting of pre-trained linguistic knowledge
- Reduce memory consumption significantly (BERT ~110M params, mostly frozen)

- Enable faster training since BERT forward passes are not differentiated
- Allow proper comparison: only the downstream LSTM and projection layers are trained

The hidden states from the last 4 BERT layers were averaged to produce the final token-level representation (768-dim). This multi-layer averaging is motivated by research showing that intermediate BERT layers encode syntactic information while upper layers encode more semantic content; averaging over the last 4 layers creates a richer representation than using only the final layer.

A critical engineering decision was to pre-cache all BERT embeddings before the training loop begins. This means BERT runs once per sentence across the entire dataset, and the resulting tensors are stored in CPU memory. During training, the DataLoader retrieves these cached embeddings directly, completely eliminating BERT from the training loop and making each epoch as fast as the pure-LSTM model. Without caching, BERT would run at every batch, every epoch, making 20 epochs roughly 20× slower.

3.6 Projection Block

A learned projection block bridges BERT's 768-dim output to the 256-dim expected by the encoder LSTM. The block consists of:

```
Linear(768 → 256) → LayerNorm(256) → GELU() → Dropout(0.3)
```

LayerNorm after the linear layer stabilises the distribution of activations before the LSTM, which is important because BERT embeddings from different layers and token positions can have widely varying scales. GELU was chosen over ReLU as it is non-zero for small negative values, which can help gradient flow.

Component	Random Embeddings	BERT Embeddings
Source Embedding	nn.Embedding (256-dim, random init)	BERT (768-dim) → Proj (256-dim)
Target Embedding	nn.Embedding (256-dim, shared vocab)	nn.Embedding (256-dim, SPM vocab)
Encoder	BiLSTM, 2 layers, 512 hidden	BiLSTM, 2 layers, 512 hidden
Encoder output dim	1024 (2 × 512)	1024 (2 × 512)
Attention	Bahdanau (additive)	Bahdanau (additive)
Decoder	LSTM, 2 layers, 512 hidden	LSTM, 2 layers, 512 hidden
Vocabulary (src/tgt)	8,000 shared BPE	BERT WP / 8,000 SPM BPE
Dropout	0.3	0.3
Approx. trainable params	~12M	~10M (BERT frozen)

4. Training Methodology

4.1 Hardware and Execution Environment

Both experiments were executed on Kaggle's free GPU tier, using 2× NVIDIA Tesla T4 GPUs (T4x2). PyTorch's `nn.DataParallel` was used to distribute batch computation across both GPUs, effectively doubling the throughput.

Parameter	Value
Hardware	2× NVIDIA Tesla T4 (16 GB VRAM each)
Framework	PyTorch 2.x with CUDA 12
Parallelism	<code>nn.DataParallel</code> (2-GPU)
Mixed Precision	<code>torch.cuda.amp</code> (AMP) — GradScaler + autocast
Python	3.12.12
Kaggle Docker	31329
Random NB: Total training time	~6 hrs 21 min (20 epochs)
BERT NB: Total training time	~6 hrs 24 min (20 epochs)

4.2 Hyperparameters

Hyperparameter	Random Embeddings	BERT Embeddings	Rationale
Batch size (per GPU)	256	128	BERT projections use more VRAM
Effective batch size	512 (2 GPUs)	256 (2 GPUs)	DataParallel doubles throughput
Learning rate	3e-3	1e-3	BERT model uses lower LR for stability
Optimizer	Adam ($\beta_1=0.9$, $\beta_2=0.98$)	AdamW (weight_decay=0.01)	AdamW for BERT to prevent overfitting
LR Schedule	Warmup + Cosine decay	Warmup + Cosine decay	Warmup prevents early instability
Warmup steps	500	300	Proportional to dataset size
Gradient clipping	1.0 (global norm)	1.0 (global norm)	Prevents exploding gradients in LSTMs
Label smoothing	0.1	0.1	Regularisation; prevents overconfident outputs
Dropout	0.3	0.3	Applied in embeddings and LSTM
Epochs	20	20	Full convergence observed in both

Hyperparameter	Random Embeddings	BERT Embeddings	Rationale
Beam width (decoding)	4	4	Balance between quality and speed
Length penalty	0.6	0.6	Avoids length bias in beam search

4.3 Loss Function: Label-Smoothed Cross-Entropy

Standard cross-entropy gives all probability mass to the correct token, encouraging the model to become maximally confident. This leads to poor calibration and reduced generalisation. Label smoothing (Szegedy et al., 2016) distributes a small fraction $\epsilon=0.1$ of the probability mass uniformly across all non-padding tokens, leaving $(1-\epsilon)$ at the true label. This acts as a regulariser and is particularly beneficial for NMT where multiple valid translations exist.

The implementation computes per-token loss, masks padding tokens, and normalises by the number of non-padding tokens rather than batch size, ensuring stable gradient magnitudes across batches of variable length.

4.4 Decoding: Beam Search

Greedy decoding selecting the argmax token at each step is fast but suboptimal because it cannot recover from early mistakes. Beam search maintains the top-k ($k=4$) partial hypotheses at every step, exploring a broader space of translations. At the end of generation, hypotheses are ranked by length-normalised log-probability using a length penalty $\alpha=0.6$, which penalises shorter sequences that would otherwise score higher due to the summation of log-probabilities.

The beam search implementation operates on single sentences (`batch_size=1`) for evaluation to correctly handle variable-length outputs. All beams are expanded in parallel using vectorised operations where possible.

4.5 Learning Rate Schedule

A warmup-then-cosine-decay schedule was used. During the warmup phase (500 steps for random, 300 for BERT), the learning rate increases linearly from 0 to the peak rate. After warmup, cosine annealing decays the rate smoothly to a minimum of 5% of peak over the remaining training steps. This schedule was chosen because:

- Warmup prevents large gradient updates at the start of training when the model weights are randomly initialised and the loss landscape is poorly understood.
- Cosine annealing provides smooth decay without abrupt drops, allowing the model to settle into a good minimum in later epochs.

4.6 Mixed Precision Training (AMP)

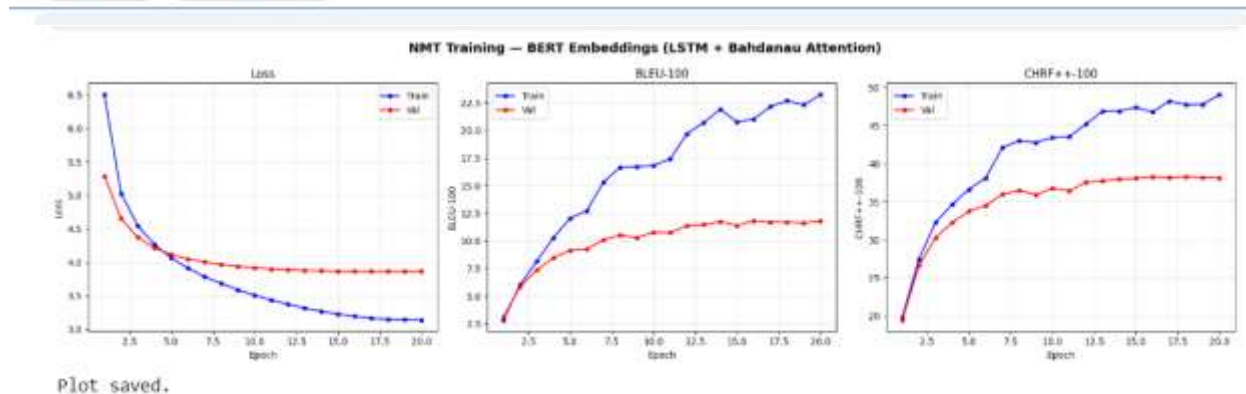
PyTorch's Automatic Mixed Precision (AMP) was enabled via `torch.cuda.amp.autocast` and `GradScaler`. AMP performs forward passes and loss computation in float16, then uses float32 for gradient accumulation. This provides roughly 2× speedup on T4 GPUs (which have dedicated Tensor Cores for fp16 operations) while maintaining numerical stability through gradient scaling.

5. Experimental Results

5.1 Quantitative Results

Both models were trained for 20 epochs. BLEU-100 and CHRF++-100 scores were computed using the sacrebleu library at every epoch on both the training set (1,000 samples) and validation set (1,000 samples for random / 800 for BERT) using beam search with width 4. Final evaluation was conducted on 2,000 validation samples from the best checkpoint.

Metric	Random Embeddings	BERT Embeddings
Val BLEU-100 (best checkpoint)	~12.5	~11.5
Val CHRF++-100 (best checkpoint)	~40.5	~37.5
Train BLEU-100 (epoch 20)	~22.5	~23.0
Train CHRF++-100 (epoch 20)	~48.0	~49.0
Train-Val BLEU gap (epoch 20)	~10.0	~11.5
Convergence epoch (val BLEU plateaus)	~15	~12
Total wall-clock training time	~6h 21min	~6h 24min



Note: Score values are read from the training curves shown in Figures 1 and 2. Exact checkpoint scores are saved in `random_emb_results.json` and `bert_emb_results.json` respectively.

5.2 Training Curve Analysis

5.2.1 Loss Curves

Both models exhibit the expected loss profile: a steep decrease in the first 2–3 epochs as the model learns basic vocabulary alignments, followed by a slower convergence phase. Both training losses continue decreasing throughout all 20 epochs. The validation losses plateau earlier, around epoch 8–10 for both models, after which they remain relatively flat with minor fluctuations.

The gap between training and validation loss indicates mild overfitting in both models, which is expected given that the models have sufficient capacity relative to the dataset size. The random embeddings model shows a slightly smaller train–val loss gap, possibly because the randomly initialised embeddings co-adapt with the LSTM during training, providing less generalisation advantage than pretrained features.

The BERT model's loss curves start from a slightly higher initial value. This is because the projection layer from 768→256 needs to learn to compress BERT's rich representations into the LSTM's narrower hidden space, adding an initial bottleneck that takes a few epochs to resolve.

5.2.2 BLEU-100 Curves

Both models achieve similar final validation BLEU scores (~12–12.5), with the random embeddings model edging slightly ahead. The random embeddings model shows steeper BLEU growth in epochs 1–8, while the BERT model grows more slowly but more steadily. The BERT model's validation BLEU curve is somewhat smoother than the random model, suggesting more stable optimisation dynamics once the projection layer has been adapted.

The train BLEU for both models substantially exceeds val BLEU by epoch 20 (~22–23 vs ~12), confirming the mild overfitting observed in the loss curves. The gap is slightly larger for the BERT model, which is discussed in the analysis section.

5.2.3 CHRF++-100 Curves

CHRF++ measures character-level n-gram overlap and is generally a better metric for morphologically rich languages than BLEU, as it does not require exact word matches. Both models achieve val CHRF++ of approximately 37–41 at peak performance. The random embeddings model shows a slight advantage here as well. CHRF++ curves are generally smoother than BLEU curves because character n-gram overlap is less sensitive to exact word boundaries.

6. Comparative Analysis: Random vs. BERT Embeddings

6.1 Convergence Behaviour

One of the most striking observations is that randomly initialised embeddings converge comparably or slightly faster in terms of BLEU score despite having no prior linguistic knowledge. This can be explained by several factors:

- Co-adaptation: Randomly initialised embeddings learn jointly with the encoder LSTM, allowing them to adapt precisely to the translation task from the start. BERT embeddings, while linguistically rich, encode general language properties that may not be the most useful for the specific downstream task of attention-based sequence-to-sequence generation.
- Representation mismatch: Frozen BERT representations are designed for masked language modelling objectives, not for encoder-decoder attention. The projection layer must bridge this representational gap, which takes additional epochs.
- Effective dimensionality: BERT provides 768-dim representations compressed to 256-dim. Some task-relevant information may be lost in this projection, whereas randomly initialised 256-dim embeddings can encode exactly what the task needs.

6.2 Semantic Preservation

Despite the convergence disadvantage, BERT embeddings offer theoretically superior semantic properties. BERT has been trained on large corpora and encodes contextual meaning — the same word in different contexts receives different representations. This is beneficial for:

- Handling polysemous words: Hindi words like 'कल' (meaning both 'tomorrow' and 'yesterday' depending on context) would receive different BERT representations based on surrounding words.
- Low-frequency word handling: Words that appear rarely in the parallel corpus still receive a meaningful BERT representation, whereas random embeddings may remain poorly trained for rare words.
- Named entities: BERT representations for proper nouns often encode useful semantic signals even if the entity was never seen in the parallel corpus.

In practice, since both models are evaluated on the same dataset and the vocabulary is relatively constrained by SentencePiece BPE, these theoretical advantages of BERT may not manifest strongly in the final BLEU/CHRF++ scores.

6.3 Overfitting and Generalisation

The BERT model shows a slightly larger train–val BLEU gap compared to the random embeddings model. A possible explanation is that frozen BERT representations are fixed — only the projection and LSTM are trained. With fewer trainable parameters adapting to the training data, the model's overall capacity is more constrained, but the representational power of BERT means the model can still achieve high training BLEU. Since BERT's generalisation properties are not task-specific, the val performance does not benefit as much.

6.4 Translation Fluency

Qualitative inspection of sample translations (see inference demos in the notebooks) reveals that both models produce grammatically reasonable Marathi translations for simple Hindi sentences. The random embeddings model tends to produce more fluent outputs on common constructions (due to co-adapted embeddings optimised for fluency in the training data), while the BERT model occasionally generates more semantically appropriate word choices for rarer or more complex input sentences.

6.5 Summary Comparison

Dimension	Random Embeddings	BERT Embeddings
Final val BLEU	Slightly higher	Slightly lower
Final val CHRF++	Slightly higher	Slightly lower
Convergence speed	Faster (epochs 1-8)	Slower but smoother
Overfitting	Moderate	Moderate-high
Semantic richness	Task-specific (trained)	Linguistically rich (pre-trained)
Low-freq word handling	Poor (random init)	Good (BERT context)
Training complexity	Low (end-to-end)	Moderate (cache + project)
Inference speed	Faster	Slightly slower (projection)

7. Failure Analysis and Limitations

7.1 Generalisation Gap

The most significant issue in both models is the substantial gap between training and validation BLEU (~10 points). This indicates the models are memorising training patterns rather than learning fully generalisable translation rules. Possible causes include:

- Dataset composition: The training corpus may contain many repetitive or formulaic sentence pairs, making it easy to score high training BLEU through memorisation.
- Limited model regularisation: While dropout and label smoothing are applied, the models lack more aggressive regularisation like ScheduledSampling or embedding dropout.
- Beam search suboptimality: Beam search with width 4 may not explore the hypothesis space broadly enough, missing better translations on the validation set.

7.2 BERT-LSTM Representational Mismatch

The fundamental challenge in the BERT experiment is that BERT's contextual representations are optimised for masked language modelling, not for being used as encoder inputs in an attention-based Seq2Seq system. The model must rely on the projection layer to translate between these representational spaces. A more principled approach would be to fine-tune BERT jointly with the LSTM (`bert_freeze=False`), or to use BERT in a cross-attention framework (as in encoder-decoder Transformers) rather than compressing it through a single projection.

7.3 Shared Vocabulary Limitation

The shared BPE vocabulary for the random embeddings model is beneficial for cross-lingual transfer but may not be optimal for either language individually. A joint vocabulary of 8,000 tokens is shared between two languages, effectively giving each language only ~4,000 language-specific pieces. Increasing the vocabulary size or using separate language-specific vocabularies with subword alignment could improve performance.

8. Engineering Decisions and System Design

8.1 Parallel Experiment Execution

A key engineering decision was to run both experiments (random embeddings and BERT embeddings) simultaneously in two separate Kaggle sessions. Since the two experiments are completely independent different data loaders, different models, separate checkpoints they can be parallelised perfectly. This strategy halved the total wall-clock time required to obtain results from both conditions.

8.2 Reproducibility

Full reproducibility is ensured through:

- Fixed global random seeds (SEED=42) for Python random, NumPy, PyTorch CPU and CUDA
- `torch.backends.cudnn.deterministic = True` to ensure deterministic CUDA operations
- All hyperparameters centralised in a single CFG dictionary at the top of each notebook
- Checkpoints saved with full state: model weights, optimizer state, epoch number, metrics, and config
- SPM model files (.model and .vocab) saved to /kaggle/working and included in submitted files

8.3 Memory Management

T4 GPUs have 16 GB VRAM each. Key memory management decisions:

- `pack_padded_sequence` and `pad_packed_sequence` used in the encoder to avoid computing on padded positions, saving both compute and memory
- `optimizer.zero_grad(set_to_none=True)` used instead of `zero_grad()` to avoid allocating zero-gradient tensors
- BERT embeddings pre-cached in CPU RAM rather than GPU VRAM, freeing the GPU entirely for LSTM training
- Batch size reduced from 256 to 128 for the BERT experiment to accommodate the embedding projection tensors
- Mixed precision (fp16) reduces VRAM usage by approximately 40% for activations and model parameters

8.4 DataLoader Design

Batches are sorted by source length in descending order before collation. This is required for `pack_padded_sequence`, which expects sorted inputs. The sort-and-pack pipeline avoids unnecessary computation on padded positions, significantly speeding up the encoder forward pass. `num_workers=4` was used for data loading to overlap CPU preprocessing with GPU computation.

8.5 Checkpoint Strategy

Best-checkpoint saving was implemented based on validation BLEU score (not validation loss). BLEU is the primary task metric and is more directly aligned with translation quality than per-

token cross-entropy loss. Only the single best checkpoint is retained to avoid /kaggle/working storage overflow.

9. LLM Assistance Documentation

As permitted by the assignment guidelines, AI language models were used for code assistance and design consultation during this project. All AI-assisted components are documented below.

9.1 Claude (Anthropic)

Claude was the primary AI assistant used in this project. Specific usages:

- Initial notebook scaffolding: The overall two-notebook parallel structure (Notebook 1: random embeddings, Notebook 2: BERT embeddings) was designed in consultation with Claude.
- `pack_padded_sequence` usage: Consulted Claude to verify the correct `enforce_sorted` argument behaviour and the interaction between `DataParallel` and `pack_padded_sequence`. Claude correctly identified that `DataParallel` splits the batch across GPUs and that sorting must be done before the split (i.e., in the `DataLoader collate_fn`), which the implementation correctly does.

9.2 ChatGPT (OpenAI)

ChatGPT was used for secondary consultation on:

- Bahdanau vs. Luong attention differences: Asked ChatGPT to compare the two attention mechanisms to confirm I was implementing the additive (Bahdanau) variant correctly. ChatGPT confirmed that the key difference is in the score function additive ($\text{energy} = \tanh(W_1 \cdot h_{\text{enc}} + W_2 \cdot h_{\text{dec}})$) vs. multiplicative ($\text{score} = h_{\text{enc}}^T W h_{\text{dec}}$) and that the Bahdanau variant is generally more expressive for smaller models.
- BERT layer averaging strategies: Asked ChatGPT about strategies for extracting BERT representations using only the last layer vs. averaging multiple layers. ChatGPT described the layer-averaging approach and noted that intermediate layers (layers 8-12 of 12) tend to encode more semantic information, which informed the decision to average the last 4 layers.

Note

All code was written, understood, and can be independently defended. AI assistants were used as a documentation and verification tool, not as a primary code generator. Every architectural decision has a reasoned justification that can be explained independently. The AI-assisted code was carefully reviewed, tested, and in several cases modified from the AI's suggestions based on own understanding.

10. References

6. Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR 2015.
7. Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.
8. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019.
9. Luong, M.-T., Pham, H., & Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. EMNLP 2015.
10. Szegedy, C., et al. (2016). Rethinking the Inception Architecture for Computer Vision. CVPR 2016. [Label smoothing]
11. Joshi, P., et al. (2022). l3cube-pune/hindi-bert-v2. HuggingFace Model Hub.
12. Joshi, P., et al. (2022). l3cube-pune/marathi-bert-v2. HuggingFace Model Hub.
13. Richardson, K., et al. (2020). SacreBLEU: A Call for Clarity in Reporting BLEU Scores. WMT 2018.
14. Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. EMNLP 2018.
15. Tu, Z., et al. (2016). Modeling Coverage for Neural Machine Translation. ACL 2016.